Diagramy sekwencji – przewodnik

Co dzieje się na każdym z 8 diagramów (wersja w Mermaidzie), skąd biorą się nazwy i na jakie pytania warto być gotowym

---

Co zmieniło się w tej wersji. Diagramy są przerysowane w Mermaidzie (bardziej czytelne, uczestnicy powtórzeni na dole). Treść jest ta sama, poza poprawkami wynikającymi z nowych plików w repo:

- **Diagram 3:** klucze Redisa zgodne z `dokumentacja_struktury_danych_redis.md` (sesja kuriera `courier:session` z TTL 60 s, blokada zamówienia `order:lock` SETNX 3 s przed zapisem kuriera). Przy odrzuceniu oferty kontroler od razu zdejmuje blokadę kuriera.
- **Diagram 1:** logowanie zgodne z dokumentacją: `validateUser` zwraca użytkownika, a tokeny generuje osobne wywołanie `generateTokens`.
- **Diagramy 4, 6, 7, 8:** dopisany `OrderParticipantGuard` (403 dla kogoś spoza zamówienia), a warunki w ramkach dotyczą już tylko statusu.
- **Diagram 5:** kurier dostaje powiadomienie o przygotowaniu, jeśli jest już przypisany. Po dostarczeniu jego sesja w Redisie jest czyszczona.

1. Jak czytać diagram sekwencji

Diagram sekwencji pokazuje **jedną konkretną sytuację** (np. złożenie zamówienia) jako rozmowę między uczestnikami. Czas płynie z góry na dół, a każda strzałka to jeden komunikat. Diagram przypadków użycia mówi, *co* system robi. Diagram sekwencji pokazuje, *jak* to robi i które moduły biorą w tym udział.

ELEMENT	ZNACZENIE
ludzik	aktor, czyli człowiek po drugiej stronie aplikacji (Klient, Dostawca, Kurier, Gość)
żółty prostokąt (powtórzony też na dole)	klasa z backendu, np. `OrdersController`, albo usługa zewnętrzna (Stripe, Nominatim, Ably)
walec	baza danych: PostgreSQL albo Redis
przerywana pionowa linia	linia życia uczestnika, na której „dzieją się" jego komunikaty
ciągła strzałka →	wywołanie: ktoś coś zleca (metoda, zapytanie SQL, żądanie HTTP)
przerywana strzałka ⇢	odpowiedź na wcześniejsze wywołanie (wynik, kod HTTP)
strzałka zawracająca do siebie	uczestnik robi coś sam, np. hashuje hasło albo generuje token
czarne kółko z numerem	kolejność kroków; w tym przewodniku odwołuję się do tych numerów
ramka **alt**	„albo–albo": wykonuje się tylko jedna z gałęzi rozdzielonych linią przerywaną, warunek jest w nawiasie
ramka **opt**	fragment wykonywany tylko wtedy, gdy warunek jest spełniony

ramka **loop**	fragment powtarzany, dopóki warunek jest spełniony
szeroki pasek ▌ WIELKIMI LITERAMI	podział diagramu na etapy (np. „REJESTRACJA" i „LOGOWANIE")
pasek z zegarem ⏱	upływ czasu (np. 15 s bez odpowiedzi kuriera)
żółta karteczka	notatka z wyjaśnieniem albo warunkiem

Dlaczego przy metodach kontrolerów nie ma `currentUser`? W dokumentacji prawie każda metoda kontrolera ma pierwszy parametr `currentUser: JwtPayload`. Nie przekazuje go użytkownik: `JwtAuthGuard` wyciąga go z tokena w nagłówku `Authorization`. Dlatego na strzałkach od aktora widać tylko to, co faktycznie wysyła aplikacja, np. `createOrder(CreateOrderDto)`.

2. Kto jest kim na diagramach

Każda klasa z diagramów należy do jednego bloku z diagramu architektury. Kontrolery to warstwa „REST API (http)", a serwisy to moduły pod nią.

UCZESTNIK NA DIAGRAMIE	BLOK W ARCHITEKTURZE	ZA CO ODPOWIADA
Klient / Dostawca / Kurier	Panel Klienta / Dostawcy / Kuriera	React (klient, dostawca), React Native (kurier)
`*Controller`	REST API (http)	przyjmuje żądania HTTP, sprawdza token i uprawnienia (guardy)
`AuthService`	Autoryzacja / Użytkownicy	rejestracja, logowanie, tokeny JWT
`OrdersService`	Zamówienia (Automat stanów)	zmiany statusu, start przydziału kuriera
`ProductsService`	Katalog	produkty i ich aktualne ceny
`GeolocationService`	Geolokalizacja i przydział kurierów	geocoding adresów, pozycje kurierów w Redisie, soft-locki
`PaymentsService`	Moduł płatności	transakcje i zwroty w Stripe, webhooki
`NotificationService`	Serwis Powiadomień	zapis do `notifications` i publikacja w Ably
`RatingService`	Oceny i opinie	zapis oceny i przeliczenie średniej
PostgreSQL / Redis	Baza danych / Cache & Geo (Redis)	dane trwałe / pozycje kurierów i blokady
Stripe, Nominatim, Ably	Zewnętrzne	płatności / zamiana adresu na współrzędne / powiadomienia w czasie rzeczywistym

3. Ustalenia, na których opierają się diagramy

- **Powiadomienia idą przez Ably**, zgodnie z dokumentacją. Backend zapisuje powiadomienie w tabeli `notifications` i publikuje je na kanale Ably, a panele odbierają je bezpośrednio z Ably.
- **Kuriera szukamy dopiero po akceptacji przez dostawcę.** Nie blokujemy kuriera dla zamówienia, które dostawca może jeszcze odrzucić. Status ACCEPTED ustawia dostawca, a kurier tylko dopisuje

się do zamówienia (`courier_id`).

- **Kurier dostaje ofertę i ma 15 s** na odpowiedź. Odrzucenie albo brak odpowiedzi oznacza, że system pyta kolejnego najbliższego kuriera i pomija tych, którzy już odmówili.
- **Redis** (klucze z `dokumentacja_struktury_danych_redis.md`): `couriers:locations` to pozycje wolnych kurierów (GEO), `courier:session:{id}` to sesja kuriera (HASH z `is_available`, `last_ping`, `current_order_id`, TTL 60 s), a `order:lock:{orderId}` to blokada zamówienia na czas zapisu kuriera (SETNX, TTL 3 s). Do tego dochodzi 15-sekundowa blokada kuriera na czas oferty z `dokumentacja_architektury.md`, nazwana tu `courier:offer:{id}`.
- **Problem z dostawą** można zgłosić tylko w trakcie dostawy (SHIPPING). Zamówienie kończy się jako `DELIVERY_FAILED`, a klient dostaje zwrot. Nie ma ponownego przydziału.
- **Zwrot pieniędzy** (`REFUNDED`) następuje przy odrzuceniu przez dostawcę, przy nieudanej dostawie i przy anulowaniu opłaconego zamówienia.

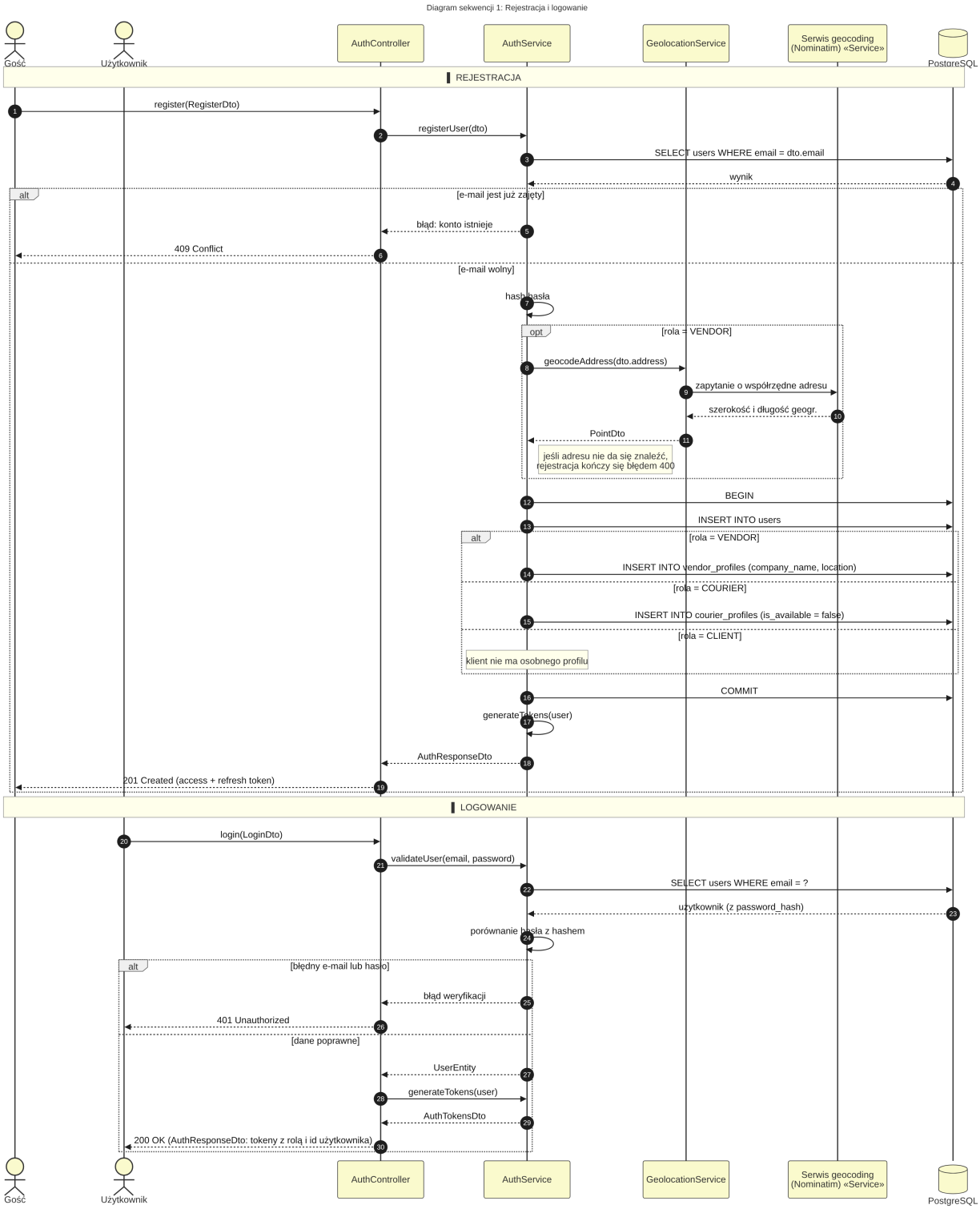


Diagram sekwencji 1 – plik sd1_rejestracja_logowanie

4. Diagram 1: Rejestracja i logowanie

Use case Zarejestruj konto (Gość), Zaloguj się (Użytkownik) Moduły REST API, Autoryzacja, Geolokalizacja, PostgreSQL, Nominatim

W skrócie: Gość zakłada konto, a system zapisuje użytkownika i, zależnie od roli, jego profil. Zalogowany użytkownik dostaje token JWT, którym potem podpisuje każde żądanie.

Krok po kroku

KROKI	CO SIĘ DZIEJE
1–4	Gość wysyła formularz (`RegisterDto`: e-mail, hasło, rola, imię, nazwisko, telefon, a dla dostawcy nazwa firmy i adres). `AuthService` sprawdza w bazie, czy e-mail jest wolny, bo kolumna `email` jest `unique`.
5–6	Jeśli e-mail jest zajęty, rejestracja kończy się kodem 409.
7	Hasło jest hashowane. W bazie trafia do `password_hash`, nigdy otwartym tekstem.
8–11	Tylko dla dostawcy: adres lokalu jest zamieniany na współrzędne przez Nominatim, bo `vendor_profiles.location` jest wymagane. Jeśli adresu nie da się znaleźć, system zwraca 400.
12–16	W jednej transakcji (BEGIN … COMMIT) powstaje wiersz w `users` i profil: `vendor_profiles` dla dostawcy albo `courier_profiles` z `is_available = false` dla kuriera. Klient nie ma osobnego profilu. Transakcja gwarantuje, że nie powstanie użytkownik bez profilu.
17–19	System generuje tokeny (access i refresh) i zwraca 201.
20–24	Logowanie: `validateUser` pobiera użytkownika po e-mailu i porównuje hasło z hashem.
25–26	Złe dane dają 401.
27–30	Dane poprawne: `validateUser` zwraca użytkownika (`UserEntity`), a kontroler prosi `generateTokens` o tokeny. Klient dostaje `AuthResponseDto` z tokenem zawierającym rolę i id użytkownika (`role`, `sub`), na podstawie którego `RolesGuard` pilnuje uprawnień.

Czemu Gość i Użytkownik to dwaj różni aktorzy?

Gość nie ma konta, więc może tylko się zarejestrować. Zalogować może się dopiero użytkownik z kontem. Tak samo jest na diagramie przypadków użycia.

Po co transakcja przy rejestracji?

Konto i profil muszą powstać razem. Jeśli zapis profilu się nie uda, wycofujemy też konto.

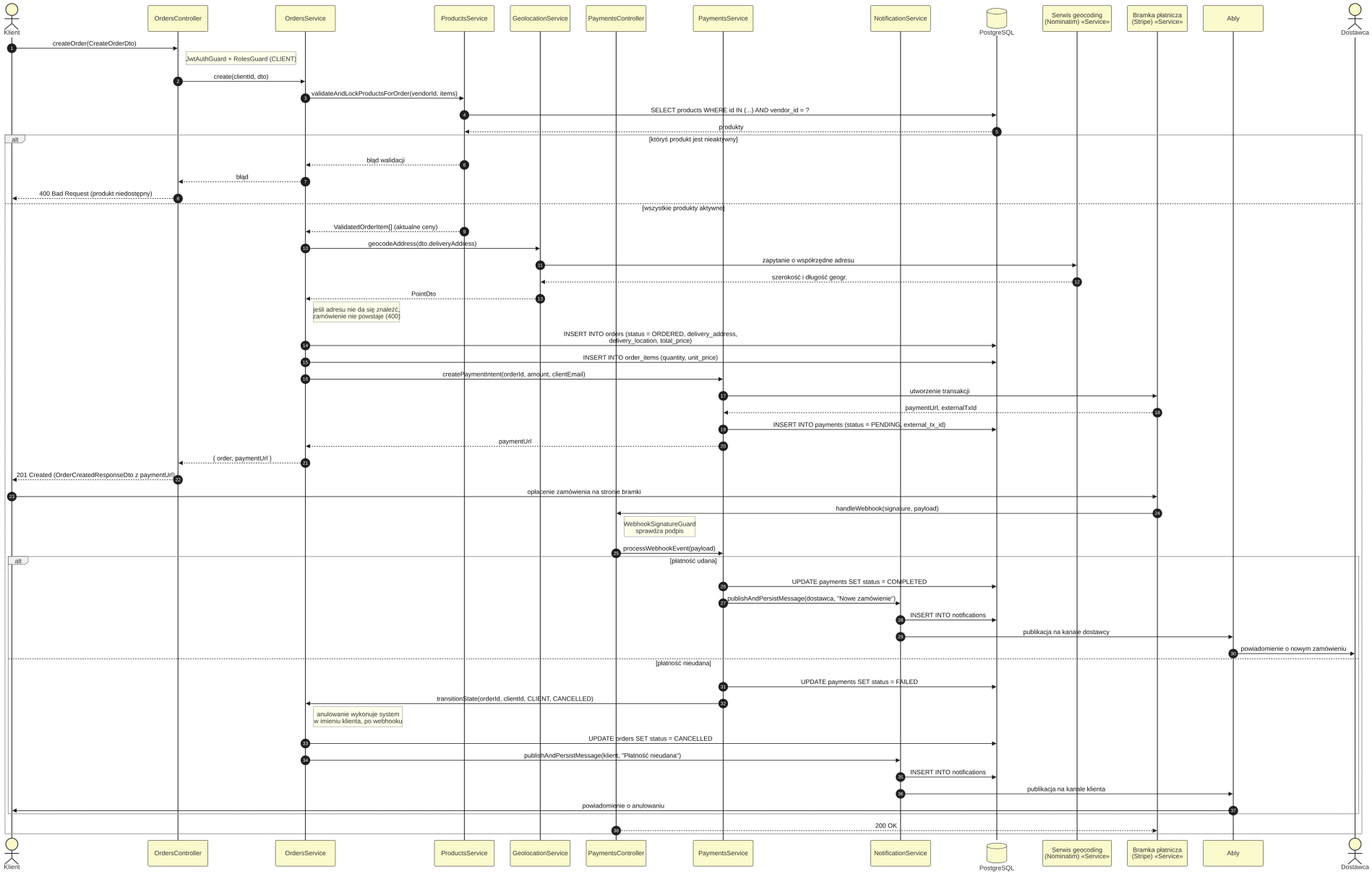


Diagram sekwencji 2 – plik sd2_zlozenie_zamowienia

5. Diagram 2: Złożenie i opłacenie zamówienia

Use case Złóż zamówienie, Podaj adres dostawy, Opłać zamówienie, Powiadom o zmianie statusu
Moduły Zamówienia, Katalog, Geolokalizacja, Moduł płatności, Serwis Powiadomień, Stripe, Nominatim, Ably

W skrócie: klient wysyła koszyk i adres. System sprawdza produkty, zamienia adres na współrzędne, zapisuje zamówienie ze statusem ORDERED i tworzy płatność w Stripe. Gdy Stripe potwierdzi wpłatę (webhook), dostawca dostaje powiadomienie o nowym zamówieniu.

Krok po kroku

KROKI	CO SIĘ DZIEJE
1–2	Klient wysyła `CreateOrderDto` (dostawca, adres dostawy, lista produktów z ilościami). Guardy sprawdzają, czy to zalogowany klient.
3–9	`ProductsService` sprawdza, czy produkty są aktywne (`is_active`), i pobiera ich **aktualne** ceny. Jeśli któryś jest ukryty, kończy się na 400 (kroki 6–8).
10–13	Adres tekstowy idzie do Nominatim, który zwraca współrzędne. Są potrzebne do śledzenia i liczenia odległości.
14–15	Zapis zamówienia: `orders` (status ORDERED, `delivery_address`, `delivery_location`, `total_price`) i pozycje w `order_items` z ceną `unit_price` „zamrożoną" w chwili zakupu.
16–22	`PaymentsService` zakłada transakcję w Stripe. Do bazy trafia płatność PENDING z `external_tx_id`, a klient dostaje link do płatności (`paymentUrl`).
23–25	Klient płaci na stronie Stripe. Stripe sam wywołuje nasz webhook, a `WebhookSignatureGuard` sprawdza podpis, żeby nikt nie podrobił „udanej płatności".
26–30	Płatność udana: status COMPLETED, zapis powiadomienia w `notifications` i publikacja w Ably. **Dopiero teraz** dostawca widzi nowe zamówienie.
31–37	Płatność nieudana: status FAILED, a zamówienie przechodzi w CANCELLED przez automat stanów. System robi to w imieniu klienta, więc jako aktor idzie `clientId` z rolą CLIENT. Klient dostaje powiadomienie.
38	Backend odpowiada Stripe kodem 200, żeby Stripe nie ponawiał webhooka.

Czemu dostawca nie dostaje powiadomienia od razu po złożeniu zamówienia?

Bo zamówienie nie jest jeszcze opłacone. Dostawca nie powinien zaczynać pracy nad czymś, za co nikt nie zapłacił.

Po co zapisywać cenę w `order_items`, skoro jest w `products`?

Dostawca może później zmienić cenę, a historia zamówień musi pokazywać kwotę z chwili zakupu.

Skąd backend wie, że płatność się udała?

Z webhooka. Stripe sam wysyła potwierdzenie na nasz serwer, a nie przeglądarka klienta, więc nie da się go podrobić.

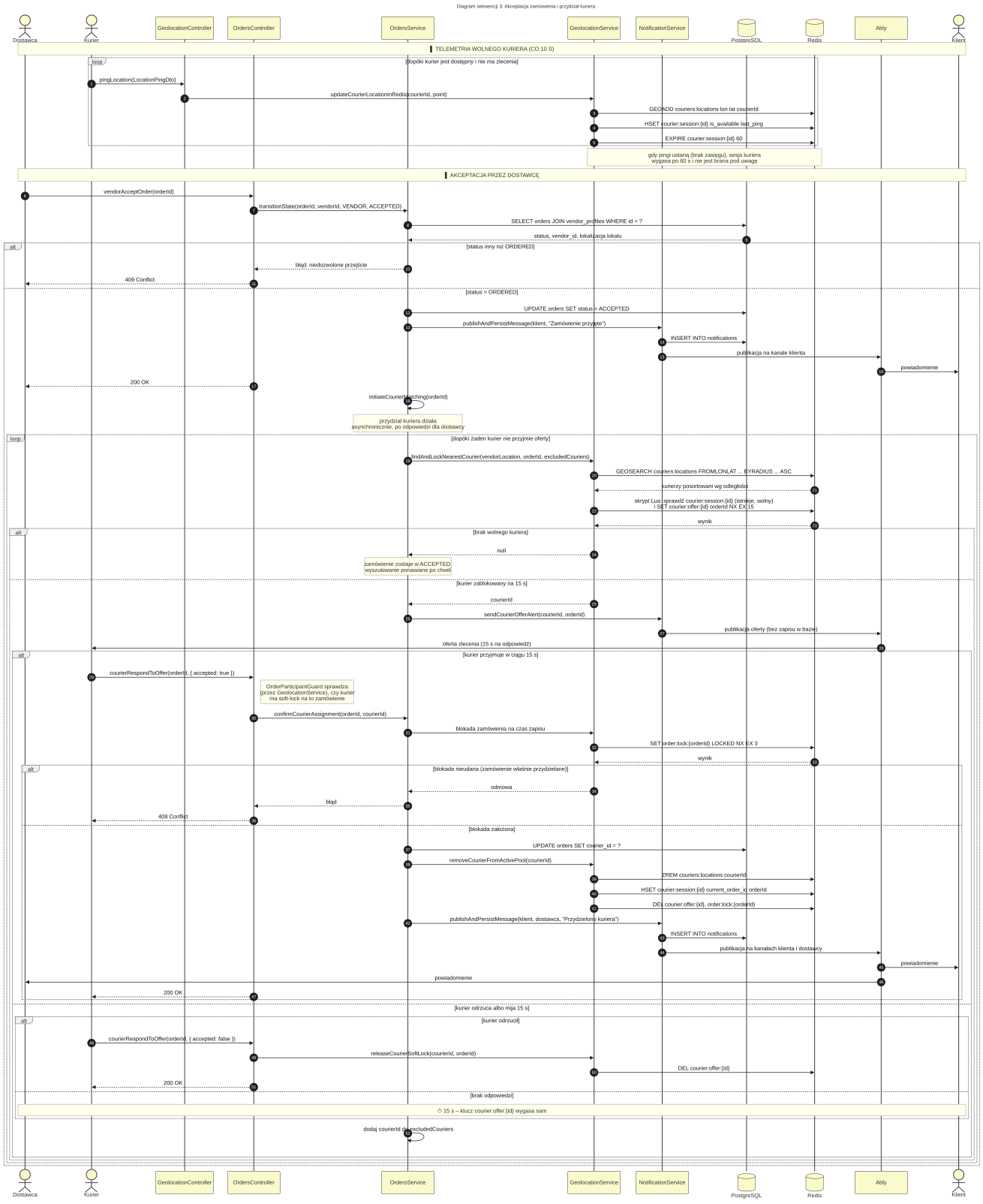


Diagram sekwencji 3 – plik sd3_akceptacja_przydzial_kuriera

6. Diagram 3: Akceptacja zamówienia i przydział kuriera

Use case Zaakceptuj zamówienie, Przydziel najbliższego dostępnego kuriera, Aktualizuj lokalizację, Odpowiedz na ofertę (Przyjmij / Odrzuć), Przydziel kolejnego kuriera Moduły Zamówienia, Geolokalizacja, Redis, Serwis Powiadomień, Ably, PostgreSQL

W skrócie: to najważniejszy i najtrudniejszy diagram projektu. Wolni kurierzy co 10 s zgłaszają pozycję do Redisa. Gdy dostawca zaakceptuje zamówienie, system szuka najbliższego wolnego kuriera, blokuje go na 15 s i wysyła mu ofertę. Jeśli kurier przyjmie ofertę, zostaje przypisany. Jeśli odmówi albo nie odpowie, system pyta następnego. Ten diagram zastępuje stary `delivery_sequence_diagram (do poprawy).pdf`.

Krok po kroku

KROKI	CO SIĘ DZIEJE
1–5	**Telemetria.** Wolny kurier co 10 s wysyła pozycję. Trafia ona tylko do Redisa: `GEOADD couriers:locations` i odświeżenie sesji `courier:session:{id}` (`HSET` i `EXPIRE 60`). Jeśli kurier straci zasięg, sesja wygaśnie po 60 s i system przestanie go brać pod uwagę (tzw. „ghost couriers"). PostgreSQL nie jest tu obciążany.
6–11	Dostawca akceptuje zamówienie. Automat stanów sprawdza, czy zamówienie jest w ORDERED i należy do tego dostawcy, oraz pobiera lokalizację lokalu. W innym przypadku zwraca 409.
12–17	Status ACCEPTED i powiadomienie klienta przez Ably. Dostawca dostaje odpowiedź 200 od razu i nie czeka na kuriera.
18	`initiateCourierMatching` uruchamia szukanie kuriera w tle (asynchronicznie).
19–23	`GEOSEARCH` zwraca kurierów posortowanych od najbliższego. Skrypt Lua **atomowo** sprawdza, czy kurier ma aktywną sesję i jest wolny, i zakłada mu blokadę oferty `courier:offer:{id}` na 15 s (`NX` , czyli tylko jeśli nikt inny jej nie założył). Dzięki temu dwa zamówienia nie dostaną jednocześnie tego samego kuriera.
24	Nie ma wolnego kuriera: zamówienie zostaje w ACCEPTED, a szukanie jest ponawiane po chwili.
25–28	Jest kandydat: oferta idzie przez Ably jako „lotny" komunikat, bez zapisu w bazie. Kurier ma 15 s na odpowiedź.
29–33	**Kurier przyjmuje.** Guard sprawdza (przez moduł Geolokalizacji), czy kurier ma blokadę oferty na to zamówienie. Przed zapisem do PostgreSQL system zakłada w Redisie blokadę zamówienia `order:lock:{orderId}` (SETNX, 3 s), jak opisuje dokumentacja Redisa.
34–36	Jeśli blokady nie udało się założyć, bo inny proces właśnie przydziela kuriera do tego zamówienia, kurier dostaje 409.
37–47	Blokada założona: twardy zapis `courier_id` w PostgreSQL, a kurier jest usuwany z puli wolnych (`ZREM`). Do sesji trafia `current_order_id` , a obie blokady są zdejmowane. Klient i dostawca dostają powiadomienie, a kurier odpowiedź 200.
48–51	**Kurier odrzuca.** Kontroler zdejmuje blokadę oferty (`releaseCourierSoftLock` , `DEL courier:offer:{id}`), więc kurier od razu jest znów wolny dla innych zamówień.
⏱	**Brak odpowiedzi.** Po 15 s klucz `courier:offer:{id}` wygasa sam.
52	W obu przypadkach kurier trafia na listę `excludedCouriers` , żeby nie dostał tej samej oferty drugi raz, a pętla zaczyna się od nowa z kolejnym najbliższym kurierem.

Po co Redis, skoro mamy PostgreSQL z PostGIS?

Setki kurierów wysyłają pozycję co 10 s. W PostgreSQL każdy taki UPDATE tworzy nową wersję wiersza (MVCC), więc baza „puchnie" i zwalnia. Redis trzyma to w pamięci i ma gotowe komendy GEO.

Czym różnią się dwie blokady?

`courier:offer:{id}` (15 s) blokuje **kuriera** na czas oferty, żeby dwa zamówienia nie zaproponowały mu pracy naraz. `order:lock:{orderId}` (3 s) blokuje **zamówienie** na moment zapisu, żeby dwa procesy nie przypisały do niego kuriera jednocześnie. Skrypt Lua i SETNX są atomowe, więc nie ma wyścigu (race condition).

Czemu szukamy kuriera dopiero po akceptacji, a nie po złożeniu zamówienia?

Dostawca może zamówienie odrzucić, a wtedy niepotrzebnie zablokowalibyśmy kuriera.

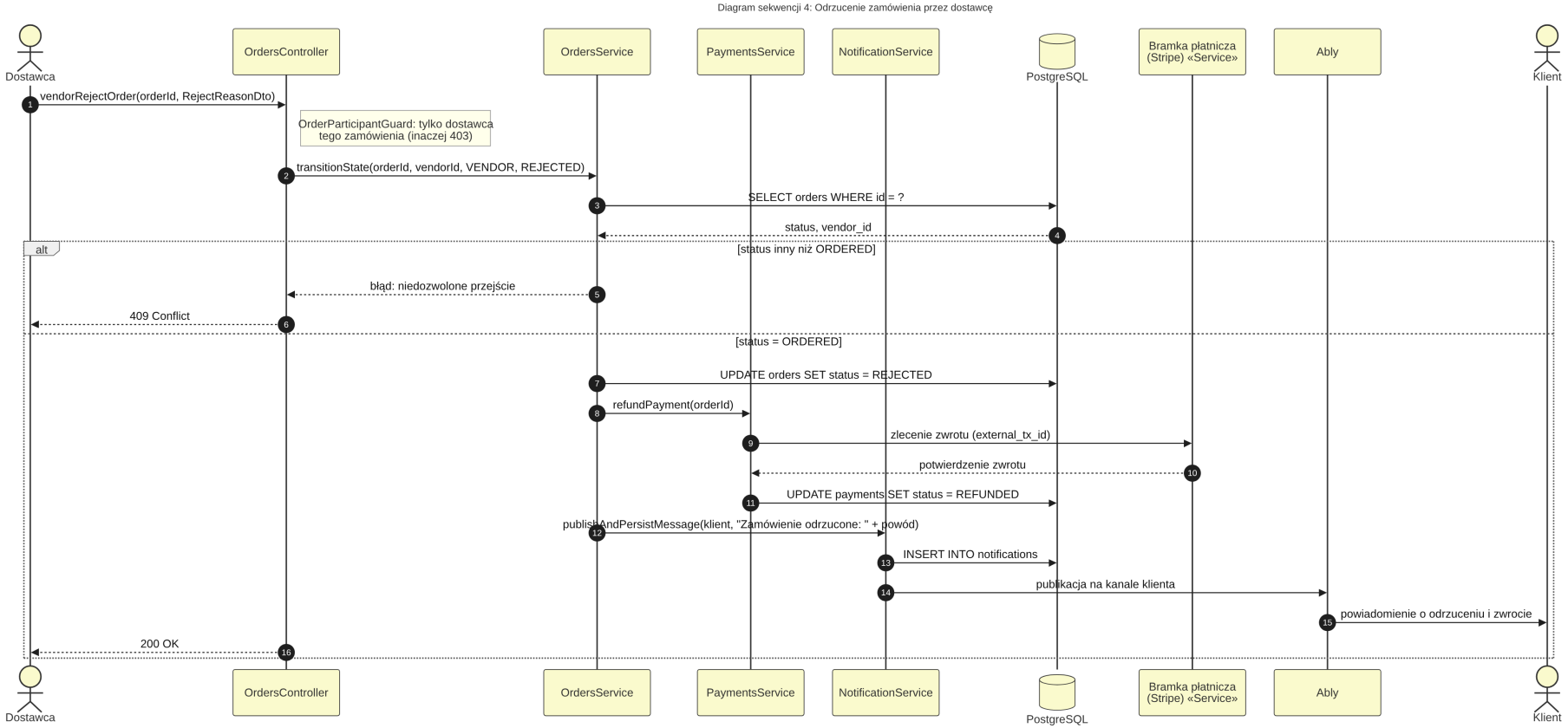


Diagram sekwencji 4 – plik sd4_odrzucenie_zamowienia

7. Diagram 4: Odrzucenie zamówienia przez dostawcę

Use case Odrzuć zamówienie, Powiadom o zmianie statusu Moduły Zamówienia, Moduł płatności, Serwis Powiadomień, Stripe, Ably

W skrócie: dostawca odrzuca nowe zamówienie z podaniem powodu (np. brak składników). Zamówienie przechodzi w REJECTED, Stripe oddaje klientowi pieniądze, a klient dostaje powiadomienie z powodem.

KROKI	CO SIĘ DZIEJE
1–6	Dostawca wysyła `RejectReasonDto` z powodem. `OrderParticipantGuard` wpuszcza tylko dostawcę tego zamówienia (inaczej 403). Automat stanów pozwala na odrzucenie tylko w ORDERED, w innym przypadku zwraca 409.
7–11	Status REJECTED. `refundPayment` zleca zwrot w Stripe po `external_tx_id`, a płatność dostaje status REFUNDED.
12–16	Powiadomienie klienta z powodem odrzucenia i odpowiedź 200 dla dostawcy.

Czy dostawca może odrzucić zamówienie, które już zaakceptował?

Nie. Odrzucenie jest możliwe tylko w ORDERED, bo po akceptacji ruszył już przydział kuriera.

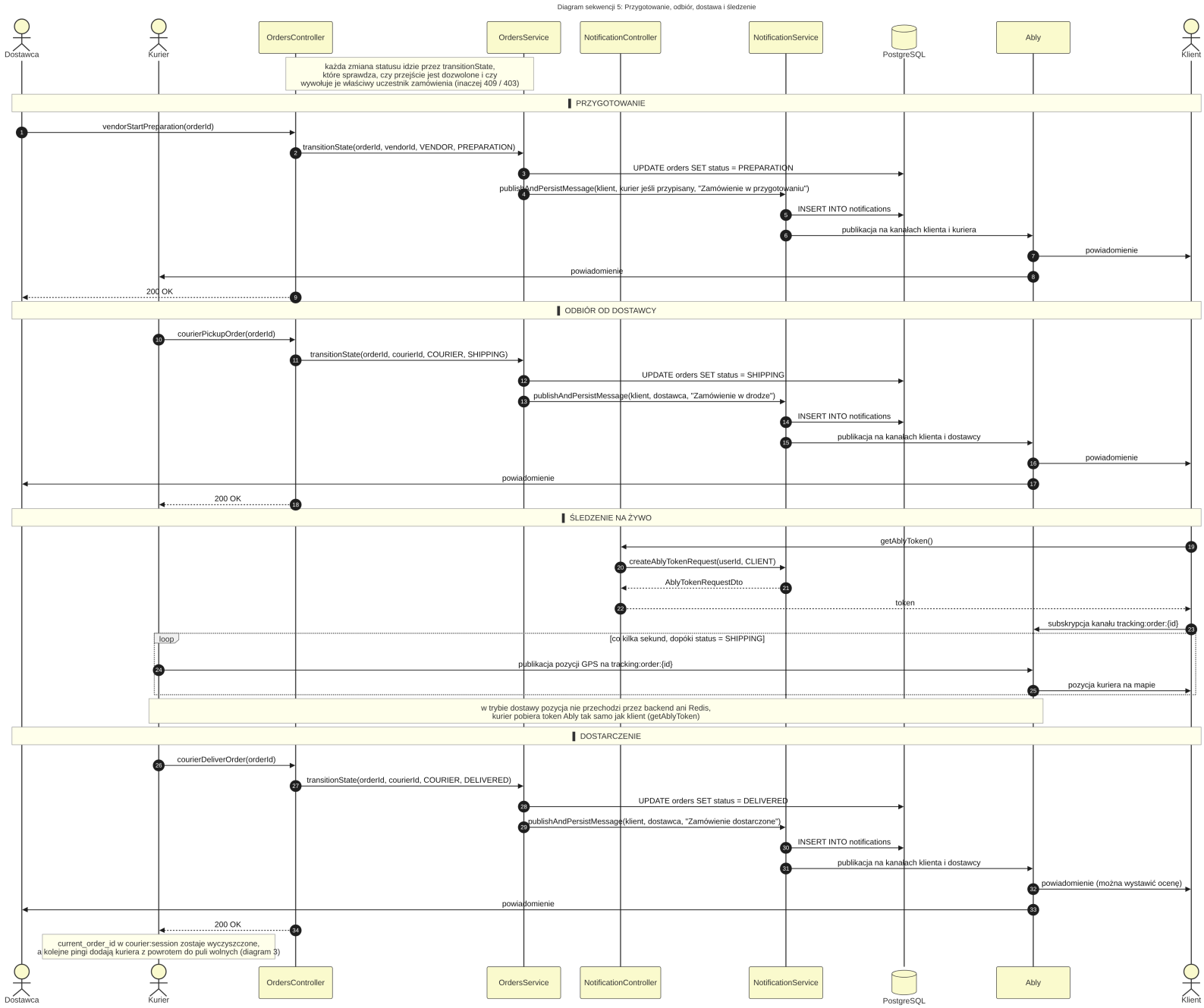


Diagram sekwencji 5 – plik sd5_realizacja_dostawa

8. Diagram 5: Przygotowanie, odbiór, dostawa i śledzenie

Use case Rozpocznij przygotowanie, Odbierz zamówienie od dostawcy, Śledź zamówienie, Potwierdź dostarczenie Moduły Zamówienia, Serwis Powiadomień, Ably, PostgreSQL

W skrócie: „szczęśliwa ścieżka" od przygotowania do doręczenia, czyli PREPARATION → SHIPPING → DELIVERED. Po każdej zmianie statusu pozostali uczestnicy dostają powiadomienie. W trakcie dostawy klient widzi kuriera na mapie na żywo.

KROKI	CO SIĘ DZIEJE
notatka	Każda zmiana statusu przechodzi przez `transitionState(orderId, actorId, actorRole, targetState)`. Ta metoda sprawdza, czy przejście jest dozwolone i czy wykonuje je właściwa osoba, a w przeciwnym razie zwraca 409 albo 403.
1–9	Dostawca zaczyna przygotowanie (ACCEPTED → PREPARATION). Powiadomienie dostają klient i **kurier**, jeśli jest już przypisany, żeby wiedział, że może jechać po odbiór.
10–18	Kurier odbiera paczkę (PREPARATION → SHIPPING). Powiadomienie dostają klient i dostawca.
19–23	Klient pobiera token Ably z backendu i subskrybuje kanał `tracking:order:{id}`. Kurier pobiera swój token tak samo.
24–25	Co kilka sekund kurier publikuje pozycję GPS **bezpośrednio do Ably**, a Ably przekazuje ją klientowi. Backend i Redis są tu pominięte, bo to tryb dostawy z dokumentacji.
26–34	Kurier potwierdza dostarczenie (SHIPPING → DELIVERED). Klient dostaje powiadomienie i od teraz może wystawić ocenę. Pole `current_order_id` w sesji kuriera jest czyszczone, a kolejne pingi dodają go z powrotem do puli wolnych (diagram 3).

Czemu pozycja w trakcie dostawy omija backend?

Bo jest potrzebna tylko klientowi tego zamówienia. Wysyłanie jej przez serwer i Redis tylko by go obciążało. Ably dostarcza ją bezpośrednio i płynnie.

Kto może zmienić status na SHIPPING?

Tylko kurier przypisany do tego zamówienia, co sprawdza `transitionState` po `actorId` i `actorRole`.

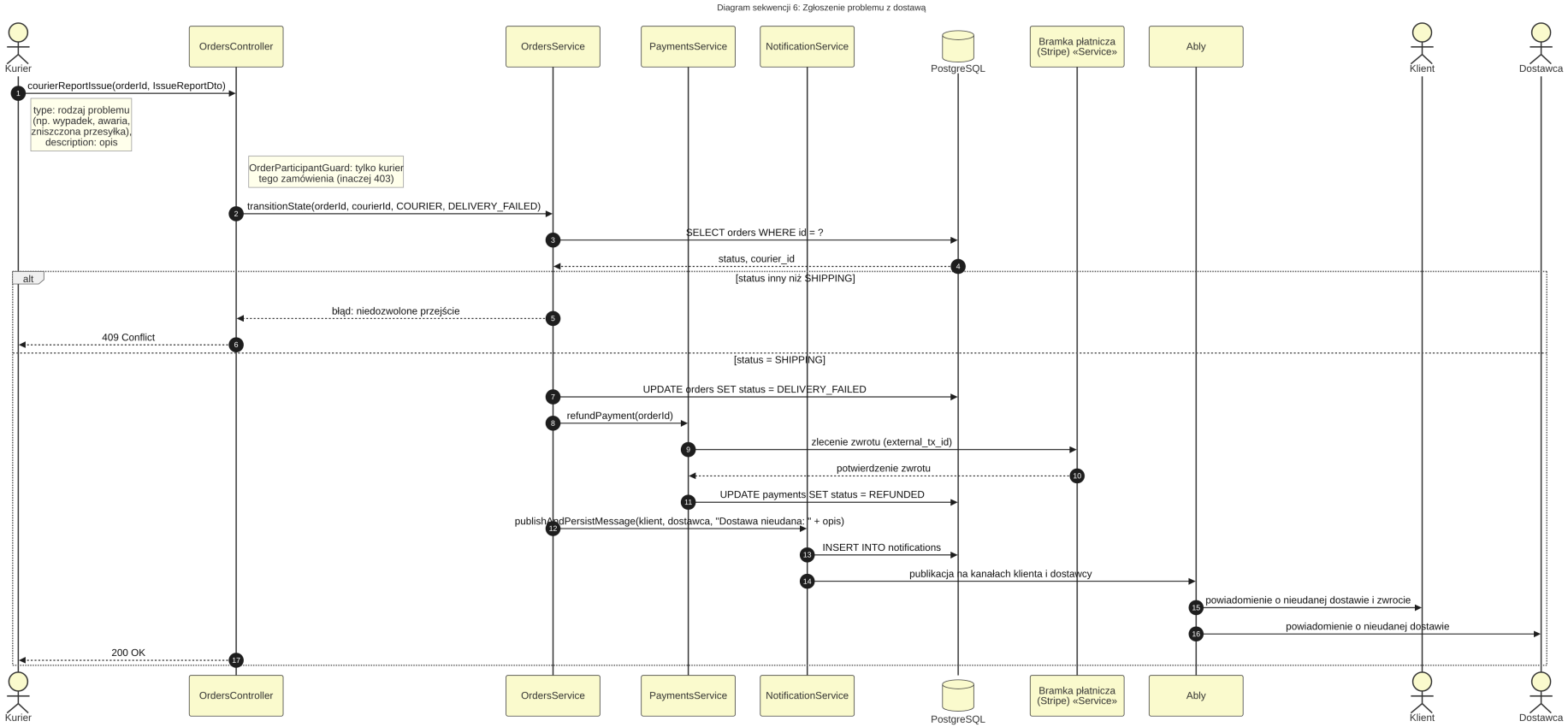


Diagram sekwencji 6 – plik sd6_problem_z_dostawa

9. Diagram 6: Zgłoszenie problemu z dostawą

Use case Zgłoś problem z dostawą, Zwróć płatność, Powiadom o zmianie statusu Moduły Zamówienia, Moduł płatności, Serwis Powiadomień, Stripe, Ably

W skrócie: w trakcie dostawy coś poszło nie tak, np. wypadek, awaria albo zniszczona przesyłka. Kurier to zgłasza, zamówienie kończy się jako DELIVERY_FAILED, klient dostaje zwrot pieniędzy, a klient i dostawca dostają powiadomienie.

KROKI	CO SIĘ DZIEJE
1	Kurier wysyła `IssueReportDto`: rodzaj problemu (`type`) i opis (`description`).
2–6	`OrderParticipantGuard` wpuszcza tylko kuriera przypisanego do zamówienia (inaczej 403). Automat stanów sprawdza, czy zamówienie jest w SHIPPING, w innym przypadku zwraca 409.
7–11	Status DELIVERY_FAILED, zwrot w Stripe i płatność REFUNDED.
12–17	Powiadomienie klienta (o nieudanej dostawie i zwrocie) i dostawcy, a potem odpowiedź 200 dla kuriera.

Czemu nie przydzielamy innego kuriera?

Tak ustaliliśmy w zespole. Przesyłka mogła zostać uszkodzona, a przekazanie jej między kurierami komplikuje proces. Najprościej i najuczciwiej wobec klienta jest zakończyć zamówienie i oddać pieniądze.

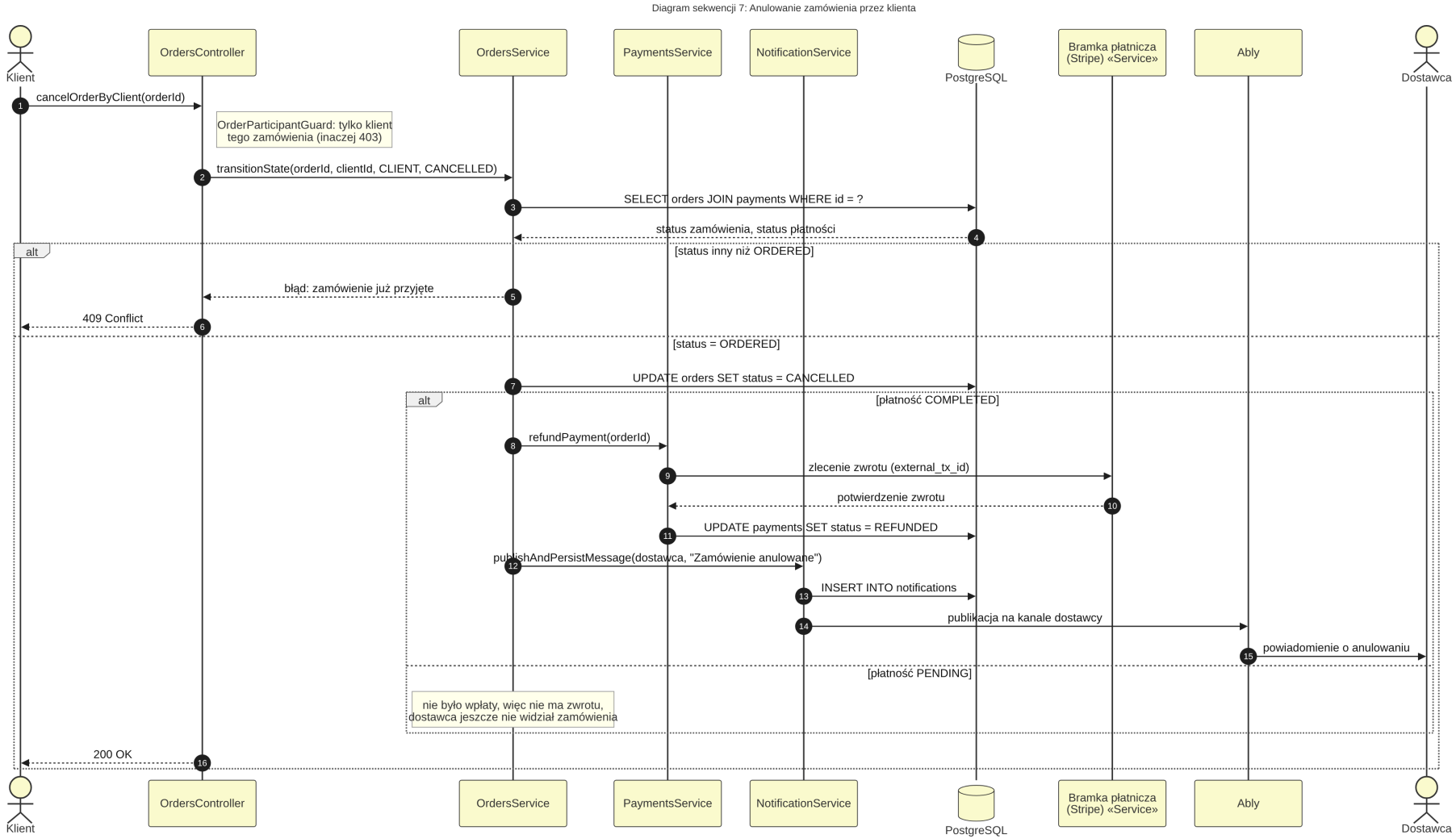


Diagram sekwencji 7 – plik sd7_anulowanie_zamowienia

10. Diagram 7: Anulowanie zamówienia przez klienta

Use case Anuluj zamówienie (extend „Śledź zamówienie", warunek: ORDERED) Moduły Zamówienia, Moduł płatności, Serwis Powiadomień, Stripe, Ably

W skrócie: klient może anulować zamówienie tylko zanim dostawca je przyjmie. Jeśli już zapłacił, dostaje zwrot, a dostawca dostaje informację, że zamówienie znika.

KROKI	CO SIĘ DZIEJE
1–6	`OrderParticipantGuard` wpuszcza tylko klienta tego zamówienia (inaczej 403). System sprawdza status zamówienia i płatności. Jeśli zamówienie nie jest w ORDERED, zwraca 409 („już przyjęte").
7	Status CANCELLED.
8–15	Jeśli płatność była COMPLETED: zwrot w Stripe, status REFUNDED i powiadomienie dostawcy, który widział to zamówienie jako nowe.
alt	Jeśli płatność była jeszcze PENDING: nie ma czego zwracać, a dostawca w ogóle nie widział zamówienia, więc nikogo nie powiadamiamy.
16	Odpowiedź 200 dla klienta.

Diagram sekwencji 8: Wystawienie oceny

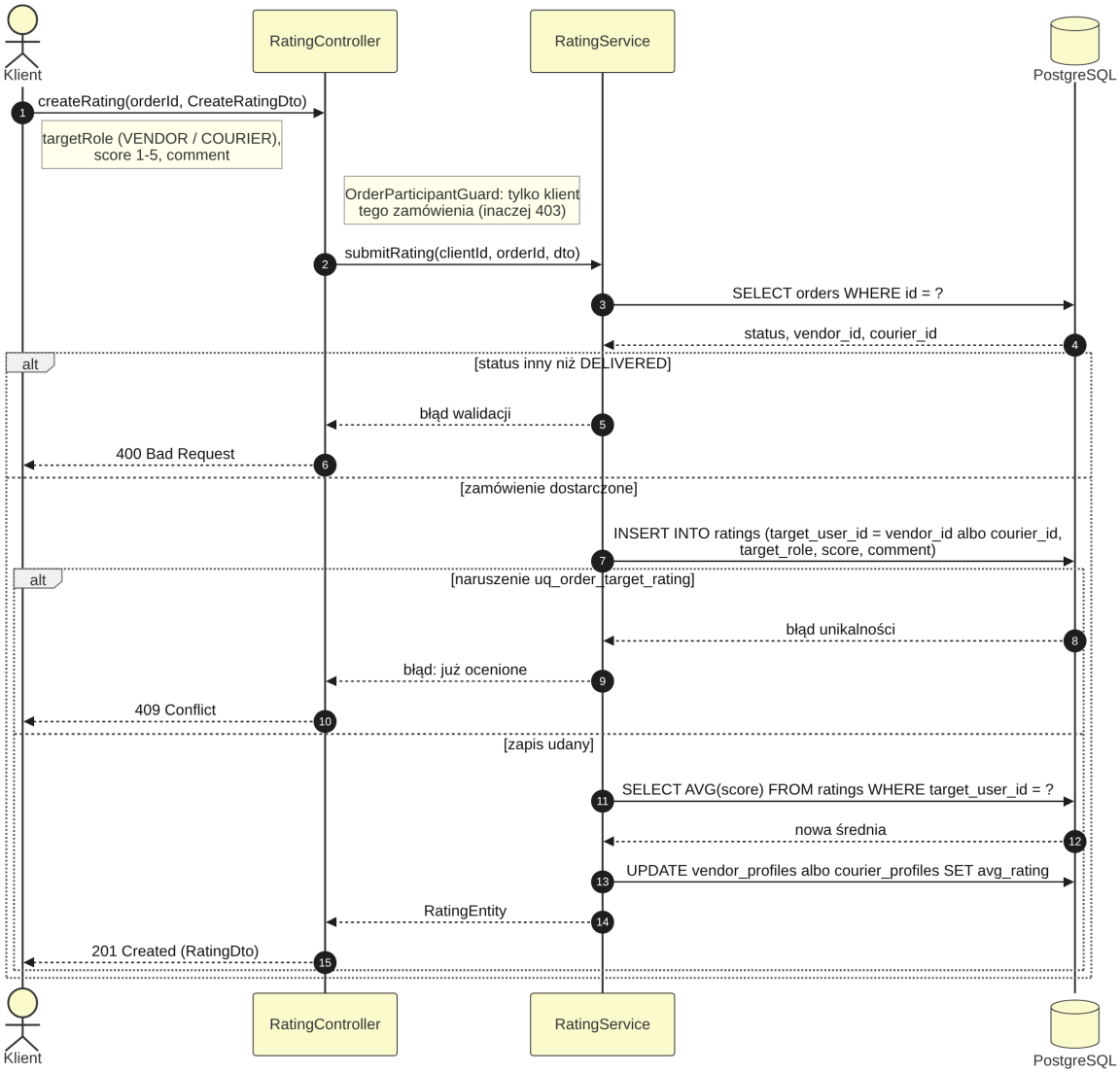


Diagram sekwencji 8 – plik sd8_ocena

11. Diagram 8: Wystawienie oceny

Use case Oceń realizację zamówienia (Oceń dostawcę / Oceń kuriera) Moduły Oceny i opinie, PostgreSQL

W skrócie: po dostarczeniu klient wystawia ocenę 1–5 z komentarzem dostawcy albo kurierowi. Każdego może ocenić raz na zamówienie. Po zapisie system przelicza średnią w profilu.

KROKI	CO SIĘ DZIEJE
1–2	Klient wysyła `CreateRatingDto`: kogo ocenia (`targetRole`: VENDOR albo COURIER), ocenę 1–5 i komentarz.
3–6	Wcześniej `OrderParticipantGuard` wpuszcza tylko klienta tego zamówienia (inaczej 403). Serwis sprawdza, czy zamówienie ma status DELIVERED. W innym przypadku zwraca 400.
7–10	Zapis w `ratings`. `target_user_id` to `vendor_id` albo `courier_id` z zamówienia. Jeśli ta osoba była już oceniona, ograniczenie `uq_order_target_rating` blokuje zapis i system zwraca 409.
11–15	Przeliczenie średniej (`AVG(score)`) i zapis w `avg_rating` profilu dostawcy albo kuriera. Na koniec odpowiedź 201.

12. Jak to się spina z resztą projektu

Przypadki użycia → diagramy sekwencji

PRZYPADEK UŻYCIA (DIAGRAMY UC1–UC4)	DIAGRAM SEKWENCJI
Zarejestruj konto, Zaloguj się	1
Złóż zamówienie, Podaj adres dostawy, Opłać zamówienie	2
Zaakceptuj zamówienie, Przydziel najbliższego dostępnego kuriera, Aktualizuj lokalizację, Odpowiedz na ofertę (Przyjmij / Odrzuć), Przydziel kolejnego kuriera	3
Odrzuć zamówienie	4
Rozpocznij przygotowanie, Odbierz zamówienie od dostawcy, Śledź zamówienie, Potwierdź dostarczenie	5
Zgłoś problem z dostawą, Zwróć płatność	6
Anuluj zamówienie	7
Oceń dostawcę, Oceń kuriera	8
Powiadom o zmianie statusu	2–7
Bez osobnego diagramu, bo to proste odczyty lub zapisy: Przeglądaj ofertę, Wyszukaj dostawców w pobliżu, Dodaj produkt do koszyka (tylko frontend), Przeglądaj historię, Zarządzaj ofertą produktów, Przeglądaj nowe zamówienia, Zmień dostępność, Przeglądaj powiadomienia	–

Co zostało sprawdzone

- **Dokumentacja (`dokumentacja_architektury.md`):** wszystkie metody i klasy z diagramów istnieją w dokumentacji, z tą samą liczbą i kolejnością argumentów, łącznie z sygnaturą `transitionState(orderId, actorId, actorRole, targetState)`. Jedyny komunikat bez nazwy metody to „blokada zamówienia na czas zapisu" (diagram 3, krok 31), bo dokumentacja jeszcze jej nie przewiduje.
- **Redis (`dokumentacja_struktury_danych_redis.md`):** klucze `couriers:locations`, `courier:session:{id}` (60 s) i `order:lock:{orderId}` (SETNX, 3 s) są użyte tak, jak opisuje dokumentacja.
- **Konwersja do Mermaida:** każdy diagram ma dokładnie te same komunikaty i ramki (alt / opt / loop) co wersja źródłowa.
- **Baza (`schema.sql`):** wszystkie tabele i kolumny z diagramów istnieją. Wszystkie statusy zamówień pochodzą z enuma `order_status`.
- **Automat stanów:** diagramy używają tylko przejść ORDERED → ACCEPTED / REJECTED / CANCELLED, ACCEPTED → PREPARATION, PREPARATION → SHIPPING, SHIPPING → DELIVERED / DELIVERY_FAILED.
- **Use-case'y:** każde `include` z diagramów UC (np. powiadomienia, zwrot, przydział kuriera) ma swój odpowiednik w diagramach sekwencji.
- **Architektura:** każda strzałka została przypisana do bloku z `final_architecture_diagram.pdf`. Pięć połączeń trzeba dorysować w architekturze (następna strona).

13. Co trzeba poprawić w repo poza diagramami sekwencji

Architektura (`final_architecture_diagram.pdf`) nie pokazuje jeszcze wszystkiego, co opisuje dokumentacja. Do dorysowania w draw.io:

- Blok **Ably** w „Zewnętrznych" oraz strzałki Serwis Powiadomień → Ably i Ably → Panel Klienta / Kuriera / Dostawcy, zamiast obecnych strzałek ws → panele. Do tego Panel Kuriera → Ably (GPS w trakcie dostawy). Nazwę „Serwis Powiadomień (ws)" zmienić na „Serwis Powiadomień".
- Serwis Powiadomień ↔ PostgreSQL (zapis i odczyt tabeli `notifications`).
- REST API → Serwis Powiadomień (token Ably, historia powiadomień, „oznacz jako przeczytane").
- Zamówienia → Moduł płatności (`createPaymentIntent`, `refundPayment`). Obecna strzałka idzie tylko w drugą stronę, więc wystarczy zrobić ją dwukierunkową.
- Zamówienia → Katalog (`validateAndLockProductsForOrder` przy składaniu zamówienia).

PLIK	CO POPRAWIĆ
`schema.sql`	W `payments` dopisać `'REFUNDED'`: `status in ('PENDING', 'COMPLETED', 'FAILED', 'REFUNDED')`. Zwroty są na diagramach 4, 6 i 7.
`dokumentacja_architektury.md`	Opis `courierReportIssue` mówi jeszcze o „ponownym szukaniu kuriera". Według ustaleń jest tylko przejście w `DELIVERY_FAILED` i zwrot pieniędzy.
`delivery_sequence_diagram (do poprawy).pdf`	Usunąć, bo zastępuje go diagram 3. Stary diagram szukał kuriera przed płatnością, używał statusu `pending`, kazał kurierowi ustawiać ACCEPTED, używał WebSocketów i innych kluczy Redisa.
`dokumentacja_struktury_danych_redis.md`	Dopisać klucz 15-sekundowej blokady kuriera na czas oferty (na diagramie `courier:offer:{id}`), bo `dokumentacja_architektury.md` jej wymaga (`findAndLockNearestCourier`, `courierRespondToOffer`), a w dokumencie Redisa jej nie ma. Poprawić też zdanie, że GEOSEARCH rusza „gdy zamówienie przechodzi w stan W przygotowaniu": według `dokumentacja_architektury.md` szukanie kuriera wyzwala akceptacja (`vendorAcceptOrder`).
`dokumentacja_architektury.md`	Dopisać w `GeolocationService` metodę do blokady zamówienia (np. `lockOrderAssignment(orderId)` z `SETNX order:lock`), żeby krok 31 na diagramie 3 miał swoją nazwę.
`README.md`	W cyklu życia zamówienia dopisać `DELIVERY_FAILED` (SHIPPING → DELIVERY_FAILED).